- `{label}`—A discriminator defined by the specific configuration data repository. In the case of Git, it can be a tag, a branch name, or a commit ID. It's useful for identifying a versioned set of config files.

Depending on your needs, you can organize the folder structure using different combinations, such as these:

```
/{application}/application-{profile}.yml
/{application}/application.yml
/{application}-{profile}.yml
/{application}.yml
/application-{profile}.yml
/application.yml
```

For each application, you can either use property files named after the application itself and placed in the root folder (e.g., /catalog-service.yml or /catalog-service-prod.yml) or use the default naming and put them in a subfolder named after the application (e.g., /catalog-service/application.yml or /catalog-service/application-prod.yml).

You can also put application.yml or application-{profile}.yml files in the root folder to define default values for all applications. They can be used as a fallback whenever there is no more specific property source. Spring Cloud Config Server will always return the properties from the most specific path, using the application name, active profiles, and Git labels.

The *label* concept is particularly interesting when using Git as the backend for the config server. For example, you could create long-lived branches of your config repo for different environments or short-lived branches while testing specific features. Spring Cloud Config Server can use the label information to return the correct configuration data from the right Git branch, tag, or commit ID.